→ Least Recently Used.

↳ LRU cache → uses DLL, Hash

LRUCache(4)

put (3,6) ~ (3,11) (2,6) (7,10) ~~(9,11)~~ ~~(4,7)~~ ~~(2,6)~~

put (4,7) ~

put (9,11) ~

put (7,10) ~ (7,10) (5,6) (3,11) (2,6)

↳ (5,6) → when adding if > capacity, delete last

get (2) ~ → 6

get (3) ~ → 11 (5,7) (7,10) (3,11) (2,6)

put (5,6) ~

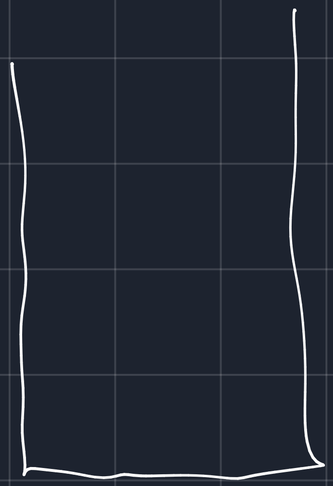
get (7) ~ → 10

put (5,7) →

↳ implementation

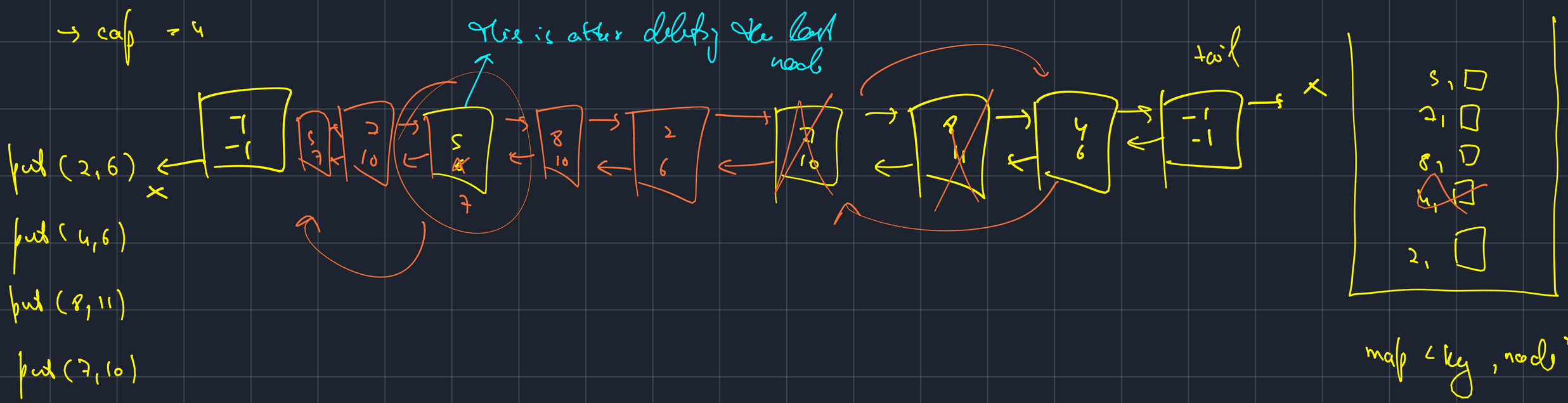
↳ DLL

↳ uses map data structure



map<key, node address>

→ cap = 4



put (2,6)

put (4,6)

put (8,11)

put (7,10)

get (2) → check in map → returns node.

get (8) → ↳ node.value

put (5,5) → exceeds capacity

get (7) ↳ delete tail → prev.

put (5,7) ↳ delete from map.

→ constructor

↳ put → exist → check the node in map → get the node → update it → rearrange in DLL

→ not exist, curr.cap < cap : insert at tail → put in map (check into the map)

→ !exist, curr.cap = cap → delete a node from the end

↳ insert the new node

& insert it at the head.